DAY 7 · PROMISES

THEORY & NOTES

Geeta University · Summer Internship 2026

2	7	5	Promises — Async Done Right
Week	Day	Topics	Focus

Topic 1 — Promise States: Pending, Fulfilled, Rejected

Ek **Promise** ek object hai jo ek asynchronous operation ke eventual result ko represent karta hai. Promise teen states mein hoti hai: **Pending** (initial state — abhi koi result nahi), **Fulfilled** (operation successful — resolve() call hua), **Rejected** (operation failed — reject() call hua). Ek baar fulfilled ya rejected ho jaaye — Promise **settled** ho jaati hai aur state KABHI nahi badlti.

Pending	Fulfilled	Rejected	Settled
Initial state. Operation chal rahi hai. Na resolved na rejected.	resolve(value) call hua. .then() ka callback value ke saath call hota hai.	reject(reason) call hua. .catch() ka callback reason ke saath call hota hai.	Fulfilled ya rejected — permanent. State dubara change nahi hogi kabhi.

Creating Promises — States & Constructor

```
// Promise constructor — executor function immediately runs (synchronously!)
const promise = new Promise((resolve, reject) => {
console.log('Executor runs NOW (sync)'); // runs immediately
setTimeout(() => {
const success = Math.random() > 0.5;
if (success) {
resolve({ id: 1, name: 'Rahul' }); // fulfills the promise
} else {
reject(new Error('Something went wrong')); // rejects
}
}, 1000);
});
console.log('Promise created:', promise); // Promise { <pending> }
// Once resolved — state is PERMANENT:
const p = new Promise((resolve, reject) => {
resolve('first'); // promise fulfilled with 'first'
resolve('second'); // IGNORED — already settled
reject('error'); // IGNORED — already settled
});
p.then(v => console.log(v)); // 'first' (only this fires)
// Promise.resolve() — shortcut to create already-fulfilled promise
const alreadyDone = Promise.resolve(42);
alreadyDone.then(v => console.log(v)); // 42
// Promise.reject() — shortcut to create already-rejected promise
```

```
const alreadyFailed = Promise.reject(new Error('fail'));
alreadyFailed.catch(e => console.log(e.message)); // 'fail'
```

■ Critical: Executor runs SYNCHRONOUSLY

new Promise((resolve, reject) => { ... }) — executor andar ka code TURANT chalata hai.

Lekin .then() callback hamesha asynchronously (microtask queue mein) call hota hai.

Ek baar resolve/reject call ho jaye — doosra call completely ignore hota hai.

Error throw karna inside executor === reject() call karna: new Promise(() => { throw err }) === rejected.

Topic 2 — .then(), .catch(), .finally()

.then(onFulfilled, onRejected) — fulfilled promise pe chalta hai. **.catch(onRejected)** — rejected promise ya chain mein koi bhi error pakadta hai. **.finally(fn)** — hamesha chalta hai (chahe resolve ho ya reject) — cleanup ke liye. Ye teeno methods ek NAYA Promise return karte hain — isliye chaining possible hai.

.then() / .catch() / .finally()

```
// .then() — fulfilled pe
Promise.resolve(10)
.then(val => {
console.log('Got:', val); // 'Got: 10'
return val * 2; // passes 20 to next .then()
})
.then(val => console.log('Doubled:', val)); // 'Doubled: 20'
// .catch() — error handle
Promise.reject(new Error('Network down'))
.catch(err => {
console.log('Caught:', err.message); // 'Caught: Network down'
return 'fallback data'; // recovers! passes to next .then()
})
.then(data => console.log('Data:', data)); // 'Data: fallback data'
// .finally() — cleanup (loading spinner stop karo, file close karo)
let isLoading = true;
fetchData()
.then(data => renderData(data))
.catch(err => showError(err))
.finally(() => {
isLoading = false; // ALWAYS runs
hideSpinner();
});
// .then() ke dono params
promise.then(
value => console.log('Success:', value), // onFulfilled
error => console.log('Error:', error) // onRejected — like .catch()
);
// NOTE: .catch(fn) === .then(undefined, fn) but .catch() handles
// errors from PREVIOUS .then() too — generally use .catch() at end
```

■ Critical: .then() vs .catch() Error Handling Difference

.then(onFulfilled, onRejected) — onRejected sirf CURRENT promise ka error handle karta hai.

.catch(fn) — promise chain mein KISI BHI jagah se aaya error handle karta hai.

Isliye: .then(fn1).catch(fn2) is BETTER than .then(fn1, fn2) for chains.

Agar fn1 mein error throw ho — .then(fn1, fn2) mein fn2 WON'T catch it, lekin .catch() will.

finally() mein return karna USELESS hai — .finally() return value ignore hoti hai.

Topic 3 — Promise Chaining

Promise chaining callback hell ka elegant solution hai. Har **.then()** ek naya Promise return karta hai — agar return value ek Promise hai to chain WAIT karta hai us promise ke settle hone ka. Agar plain value return karo — wo automatically fulfilled promise mein wrap hoti hai.

Promise Chaining — Sequential Async

```
// ■■ PROMISE CHAINING — linear, readable ■■■■■■■■■■■■■■■■■■■■■■■■
function getUser(id) {
return new Promise(resolve =>
setTimeout(() => resolve({ id, name: 'Rahul', teamId: 5 }), 300)
);
}
function getTeam(teamId) {
return new Promise(resolve =>
setTimeout(() => resolve({ id: teamId, name: 'Frontend Team' }), 200)
);
}
function getProjects(teamId) {
return new Promise(resolve =>
setTimeout(() => resolve(['App v2', 'Dashboard', 'API']), 250)
);
}
// Chain them — each .then() waits for previous to complete
getUser(1)
.then(user => {
console.log('User:', user.name);
return getTeam(user.teamId); // returns Promise — chain waits
})
.then(team => {
console.log('Team:', team.name);
return getProjects(team.id); // returns Promise — chain waits
})
.then(projects => {
console.log('Projects:', projects.join(', '));
})
.catch(err => {
console.error('Chain broke:', err.message); // catches ANY error above
});
```

```
// Compare with callback hell equivalent (6+ levels of nesting!)
```

Chaining Rules + Common Mistake

```
// ■■ CHAINING RULES ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Rule 1: Return value → next .then() gets it
Promise.resolve(1)
.then(v => v + 1) // returns 2
.then(v => v * 3) // returns 6
.then(v => console.log(v)); // 6
// Rule 2: Return Promise → chain waits for it
Promise.resolve('start')
.then(v => new Promise(res => setTimeout(() => res('waited!'), 500)))
.then(v => console.log(v)); // 'waited!' after 500ms
// Rule 3: Throw error → jumps to next .catch()
Promise.resolve('ok')
.then(v => { throw new Error('oops'); })
.then(v => console.log('skipped')) // NEVER runs
.catch(e => console.log('caught:', e.message)) // 'caught: oops'
.then(() => console.log('chain continues')); // runs after catch recovers
// Rule 4: .catch() can RECOVER — returns fulfilled promise
Promise.reject('error')
.catch(e => 'recovered!') // returns fulfilled promise with 'recovered!'
.then(v => console.log(v)); // 'recovered!'
// COMMON MISTAKE: Forgetting to RETURN inside .then()
getUser(1)
.then(user => {
getTeam(user.teamId); // ■ no return! next .then() gets undefined
})
.then(team => console.log(team)); // undefined — bug!
```

Topic 4 — Promise.all(), allSettled(), race(), any()

Jab multiple async operations ko handle karna ho to Promise static methods use karte hain. **Promise.all()** — sab fulfill hone par resolve, koi bhi reject hone par turant reject. **Promise.allSettled()** — sab complete hone ka wait karta hai, koi fail ho tab bhi. **Promise.race()** — jo pehle settle ho (resolve ya reject) — wahi result. **Promise.any()** — pehla jo FULFILL ho (rejections ignore karta hai).

Promise.all / allSettled / race / any — Comparison

```
const p1 = new Promise(res => setTimeout(() => res('P1 done'), 300));
const p2 = new Promise(res => setTimeout(() => res('P2 done'), 100));
const p3 = new Promise(res => setTimeout(() => res('P3 done'), 200));
const pFail = new Promise((_, rej) => setTimeout(() => rej(new Error('P4 failed')), 150));
// ■■ Promise.all() — ALL must succeed ■■■■■■■■■■■■■■■■■■■■■■■■■
Promise.all([p1, p2, p3])
.then(results => console.log(results));
// ['P1 done', 'P2 done', 'P3 done'] — ORDER PRESERVED (not arrival order)
// Takes ~300ms (longest promise)
```

```
Promise.all([p1, pFail, p3])
.then(results => console.log(results)) // never runs
.catch(err => console.log('Failed:', err.message));
// 'Failed: P4 failed' at ~150ms — FAIL FAST!
// ■■ Promise.allSettled() — ALL complete (fail or not) ■■■■■■■■■
Promise.allSettled([p1, pFail, p3])
.then(results => {
results.forEach(r => {
if (r.status === 'fulfilled')
console.log('✓', r.value);
else
console.log('✗', r.reason.message);
});
});
// ✓ P1 done
// ✗ P4 failed
// ✓ P3 done
// (takes ~300ms — waits for ALL)
// ■■ Promise.race() — FIRST to settle (win or lose) ■■■■■■■■■■■
Promise.race([p1, p2, p3])
.then(winner => console.log('Race winner:', winner));
// 'Race winner: P2 done' (~100ms — fastest)
Promise.race([p1, pFail])
.catch(err => console.log('Race lost:', err.message));
// 'Race lost: P4 failed' (~150ms — pFail settles first)
// ■■ Promise.any() — FIRST to FULFILL (ignores rejections) ■■■■■
Promise.any([pFail, p2, p3])
.then(first => console.log('First success:', first));
// 'First success: P2 done' (pFail rejected — ignored!)
// If ALL reject: AggregateError
Promise.any([Promise.reject('a'), Promise.reject('b')])
.catch(e => console.log(e instanceof AggregateError, e.errors));
// true, ['a', 'b']
```

■ Kab Kya Use Karein?

Promise.all() — parallel API calls jo SABHI chahiye (user + posts + comments ek saath).

Promise.allSettled() — jab partial success OK ho (batch operations, analytics events).

Promise.race() — timeout implement karo: Promise.race([fetch(url), timeout(5000)]).

Promise.any() — multiple mirrors/fallbacks: pehla jo respond kare wo use karo.

Topic 5 — Converting Callbacks to Promises

Legacy code aur Node.js built-in APIs callbacks use karte hain. Inhe Promise-based APIs mein convert karna — **Promisification** — cleaner async code ke liye zaroori skill hai. Node.js ka **util.promisify()** automatically convert karta hai error-first callbacks ko. Custom promisification bhi seekhna zaroori hai.

Callback API

```
// Callback-based API
function readFile(path, encoding, cb) {
fs.readFile(path, encoding, (err, data) => {
if (err) cb(err);
else cb(null, data);
});
}
// Usage — callback style
readFile('./data.txt', 'utf8',
(err, data) => {
if (err) { console.error(err); return; }
console.log(data);
}
);
```

Promisified Version

```
// Promisified version
function readFileP(path, encoding) {
return new Promise((resolve, reject) => {
fs.readFile(path, encoding, (err, data) => {
if (err) reject(err);
else resolve(data);
});
});
}
// Usage — promise style
readFileP('./data.txt', 'utf8')
.then(data => console.log(data))
.catch(err => console.error(err));
// Or with async/await (Day 8)
const data = await
readFileP('./data.txt','utf8');
```

Generic Promisify + Real-World Examples

```
// ■■ GENERIC PROMISIFY FUNCTION ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
function promisify(fn) {
return function(...args) {
return new Promise((resolve, reject) => {
fn(...args, (err, result) => { // adds callback at end
if (err) reject(err);
else resolve(result);
});
});
};
}
// Convert any Node-style callback function
const readFileP = promisify(fs.readFile);
const writeFileP = promisify(fs.writeFile);
const statP = promisify(fs.stat);
readFileP('./config.json', 'utf8')
.then(data => JSON.parse(data))
.then(config => console.log('Config:', config))
.catch(err => console.error('Error:', err.message));
// ■■ Node.js built-in util.promisify ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
const { promisify: nodePromisify } = require('util');
const readFileNode = nodePromisify(fs.readFile);
// Same result — Node does the same thing internally
// ■■ Real-world: Promisify setTimeout ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
const delay = ms => new Promise(resolve => setTimeout(resolve, ms));
// Usage
delay(1000).then(() => console.log('1 second passed!'));
```

```
// In async/await (Day 8 preview):
async function retryWithDelay(fn, retries, delayMs) {
for (let i = 0; i < retries; i++) {
try { return await fn(); }
catch (e) {
if (i === retries - 1) throw e;
await delay(delayMs * (i + 1)); // exponential backoff
}
}
}
// ■■ getUserMedia — browser API promisification ■■■■■■■■■■■■■■■■■■
function getCameraPermission() {
return navigator.mediaDevices.getUserMedia({ video: true, audio: false })
.then(stream => { console.log('Camera access granted'); return stream; })
.catch(err => { throw new Error(`Camera denied: ${err.message}`); });
}
```

■ Retry with Delay — Production Pattern

delay() helper — delay(ms) = new Promise(res => setTimeout(res, ms)) — ek line, pure gold.

Async/await ke saath: await delay(1000) — ek second ruko, bilkul readable.

Retry with backoff: delay(baseDelay * 2^attempt) — network errors ke liye standard pattern.

Node.js 'timers/promises': import { setTimeout as delay } from 'timers/promises' — built-in!

Day 7 Learning Outcomes

Promise ke teen states aur lifecycle confidently explain kar paoge.

.then()/.catch()/.finally() chain likhna aayega — error recovery bhi.

Promise chaining se sequential async operations handle kar paoge.

Promise.all/allSettled/race/any ka correct use case pata hoga.

Callback-based functions ko Promises mein convert kar paoge — generic promisify bhi.

Retry with delay pattern implement kar paoge.